Missing Rate Limiting on Lead Creation Endpoint – Unbounded Request Processing

Severity: High

CVSS Score: 7.5 (High)

(Note: This score reflects a theoretical worst-case availability impact. An active high volume & high frequency DoS testing was out of scope and not performed. The score assumes the impact if the endpoint is exploited at a real world attack scale.)

CWE

- CWE-799: Improper Control of Interaction Frequency
- CWE-770: Allocation of Resources Without Limits or Throttling
- CWE-400: Uncontrolled Resource Consumption

Affected Asset

- URL: https://crm-stage.crusource.com
- Endpoint: POST /api/v1/leads
- Environment: Staging
- Date Discovered: 14-10-2026

Why this is a vulnerability:

- The lead creation endpoint (POST /api/v1/leads) does not enforce any form of rate limiting, throttling, or request frequency control. The server processes every submitted request immediately and returns a success response, regardless of how many requests are sent in a short window.
- During testing, 20 sequential create-lead requests were submitted via Burp Suite Intruder within approximately 30 seconds. Every request returned **HTTP/2 201 Created**. No 429 Too Many Requests responses were returned, no Retry-After headers were present, and no progressive delays or challenges were observed.
- The endpoint performs substantial server-side work per request: input validation, database writes, business rule evaluation, integration triggers, and analytics updates. Each request consumes CPU, memory, and database connection capacity. Without a throttle, these resources can be consumed without limit by an authenticated user.
- The application also exposes a CSV bulk-import feature for leads, indicating that batch create operations are a first-class capability. Despite this, the single-record create

endpoint remains unthrottled – and single-record endpoints are the easiest to abuse programmatically since a script can send hundreds of requests per second, each triggering the full backend pipeline.
- In a multi-tenant CRM, one tenant's abuse can degrade service for all tenants sharing the same infrastructure. The absence of rate limiting removes the primary defense against this scenario.

Delivery vector (how an attacker would exploit it):

- **Automated Request Flooding:** An authenticated user can write a simple script (curl loop, Python requests, Burp Intruder) to submit the create-lead request at high frequency. Because no throttle exists, every request is processed by the server without delay, rejection, or challenge.
- **Resource Exhaustion:** Each request consumes database write capacity and application memory. In sufficient volume, this can exhaust connection pools, drive up CPU utilization, and slow or halt processing for other tenants or users on the same infrastructure.
- **Accidental Abuse via Misconfigured Integration:** A marketing automation tool, webhook retry loop, or buggy import script could generate the same effect without malicious intent. Rate limiting is the standard safeguard against this class of accident – its absence means the platform has no protection against self-inflicted overload.
- **Cost Exploitation:** If the CRM is billed by record count, API call volume, or third-party integration calls, automated request submission directly translates to increased operational cost for the tenant.
- **Combined with the Deduplication Finding:** Because the endpoint also lacks server-side deduplication (reported separately), a single script can generate thousands of unique, fully-operable lead records in seconds – each one consuming storage, indexing, integration syncs, and downstream processing. This magnifies the availability impact beyond simple request flooding.
- **Scope Note:** Active DoS testing was **not performed** and is outside the approved engagement scope. Automated volume testing was deliberately limited to 20 requests to observe the endpoint's behavior without causing service degradation. The severity rating reflects the theoretical availability impact if the missing control is exploited at scale, not a demonstrated outage.

Remediation recommendation

1. **Enforce duplicate detection on the server.** Before creating a new lead, query the database for an existing record matching the uniqueness key (e.g., email, phone, or a composite of name + company). Return 409 Conflict with the existing record's ID if a match is found.
2. **Require explicit confirmation for legitimate overrides.** If "create anyway" is a supported workflow, the server must require a dedicated confirmation parameter that the

UI sets only after user interaction. For example, POST /api/v1/leads with "confirm_duplicate": true.

3. **Add idempotency keys.** For all create endpoints, accept a client-supplied Idempotency-Key header. Cache successful responses keyed by that value for a short window (e.g., 24 hours) and return the original response if the same key is reused. This blocks replay-style duplication regardless of payload content.
4. **Reconcile existing duplicates.** Audit the current lead database for accidental or malicious duplicates and provide an admin-side merge tool. Deleting duplicates silently may destroy attached notes, activities, or history.
5. **Log and alert on anomalous create volume.** Emit a security event when a single user or API key creates an unusually high number of leads in a short window; alert tenant administrators.

Summary

During a controlled test of 20 sequential create requests submitted via Burp Suite Intruder, every request returned HTTP/2 201 Created with no 429, no Retry-After header, and no progressive delay. The server processes every submitted request regardless of frequency, exhausting database write capacity, memory, and integration quotas on each invocation. While active DoS testing was out of scope and no outage was demonstrated, the absence of this control removes the primary defense against automated request flooding, accidental bulk submissions from misconfigured integrations, and resource exhaustion – all of which are standard risks for ‘create’ endpoints on multi-tenant SaaS platforms. Combined with the separately reported missing server-side deduplication on the same endpoint, an authenticated user can generate an unbounded volume of persistent, fully-operable duplicate lead records via simple request replay or scripting. Remediation requires implementing per-user and per-IP rate limiting, applying tighter limits to authentication endpoints, adding dedicated limits to the CSV import feature, and emitting security alerts on threshold breaches.